

Polars Playground

v1.12.0

Plataforma web interactiva para aprender Polars de forma práctica

Estado: En producción | Última actualización: Septiembre 2026 | Ejecución: Client-side (WASM / Pyodide)

Resumen del Proyecto: El usuario escribe código Python directamente en el navegador y lo ejecuta contra tests automáticos en un entorno seguro e interactivo.

Tecnologías Principales

Tecnología	Versión	Función y Descripción
HTML5 & CSS3	-	Estructura base, estilos customizados y diseño responsive.
JavaScript vanilla	-	Lógica global de la aplicación, manipulación del DOM y orquestación.
Bootstrap	v5.3.3	Navegación mobile responsive, sistema de pestañas (tabs) y layout.
Pyodide	v0.27.7	Entorno de ejecución de Python basado en WebAssembly (WASM).
Monaco Editor	v0.45.0	Editor de código interactivo con syntax highlighting (motor de VS Code).
Polars	latest	Librería de alto rendimiento para análisis y manipulación de datos.
Service Worker	v5	Caché de archivos estáticos con estrategia <i>network-first</i> para uso offline.

Estructura del Proyecto

Organización de archivos y módulos principales de la plataforma:

```
Polars Playground/
├── index.html           # Página principal / Landing
├── principiante.html    # Grid de lecciones principiante
├── intermedio.html      # Grid de lecciones intermedio
├── avanzado.html        # Grid de lecciones avanzado
├── desafio.html         # Desafío final (3 retos)
├── lecciones_principiante.html # Playground principiante (14 lecciones)
├── lecciones_intermedio.html  # Playground intermedio (13 lecciones)
├── lecciones_avanzado.html    # Playground avanzado (13 lecciones)
├── oso_polar.webp        # Logo del oso polar (header + favicon + OG image)
├── png_polars.svg        # Logo oficial de Polars
├── robots.txt           # Reglas para motores de búsqueda
├── sitemap.xml          # Mapa del sitio para indexación
├── sw.js                # Service Worker v5 ( network-first )
├── vercel.json          # Configuración de despliegue en Vercel
├── .vercelignore        # Exclusiones de despliegue
├── requirements.txt     # Dependencias Python
└── README.md           # Esta documentación
```

Niveles de Aprendizaje (40 Lecciones + 3 Desafíos)

Principiante 14 Lecciones Importar, DataFrame, CSV, select, filter, expresiones, estadísticas, sort, with_columns, group_by, nulls, write_csv y proyecto final.	Intermedio 13 Lecciones Joins (inner/left/right/outer/cross), concat, pivot, unpivot, fechas (.dt), date_range, Lazy API y optimización.	Avanzado 13 Lecciones Collect con filtros, window functions (.over), rank, shift/diff, rolling windows, SQL en Polars, when/then, categorical y explode.	Desafío Final 3 Retos Prácticos Limpieza de Datos Compleja, Análisis Aumentado de Ventas y Pipeline Completo de Datos.
--	---	---	---

Funcionalidades Principales

Editor & Panel de Salida • Monaco Editor: Syntax highlighting para Python, números de línea, autocompletado y atajos (<code>Ctrl+Enter</code> para ejecutar, <code>Ctrl+S</code> para solución). • Jupyter-style Output: Renderizado de tablas HTML vía <code>df.to_dict()</code> + constructor JS, evitando conflictos de CSS de <code>df.write_html()</code>. • Detección automática de DataFrames en el scope y filtrado inteligente de warnings (Warning, DeprecationWarning, FutureWarning).	Sistema de Tests Automáticos • Validaciones en tiempo real tras la ejecución del código. • Checklist Visual: Resumen claro con tests individuales pasados/fallidos en lugar de tracebacks crudos. • Manejo directo de errores al inicio con lista detallada y contador de aciertos.
Guía Educativa & Progreso • Modal de Guía: Manual integrado accesible con "📖 Guía de Funciones", adaptado dinámicamente según el nivel. • Persistencia: Progreso guardado automáticamente en <code>localStorage</code> por categoría.	Adaptabilidad Mobile & Header • Bootstrap 5.3.3 con vista optimizada en 3 pestañas (Instrucciones, Editor, Resultado). • Logo de oso polar adaptativo con filtro CSS <code>brightness(0) invert(1)</code> e indicadores visuales de avance por nivel.

Almacenamiento Local (localStorage)

Clave (Key)	Descripción del Contenido Almacenado
<code>polarsCompleted</code>	Array de lecciones completadas del Nivel Principiante

Clave (Key)	Descripción del Contenido Almacenado
<code>polarsCompletedIntermedio</code>	Array de lecciones completadas del Nivel Intermedio
<code>polarsCompletedAvanzado</code>	Array de lecciones completadas del Nivel Avanzado
<code>polarsDesafioCompleted</code>	Array de retos globales completados en el Desafío Final

Instalación y Despliegue

Al ser un proyecto **100% estático**, no requiere compilación en servidor. Para desarrollo local:

```
# Opción 1: Python
python -m http.server 8000

# Opción 2: Node.js
npx serve .
```

Nota: Pyodide y Service Workers requieren protocolo HTTP/HTTPS (no funcionan directamente con `file://`).

Estrategia SEO & Optimización

Página	Título SEO	Robots Meta Tag
<code>index.html</code>	Polars Playground - Aprende Polars de Forma Interactiva	<code>index, follow</code>
<code>principiante.html</code>	Nivel Principiante - Polars Playground	<code>index, follow</code>
<code>intermedio.html</code>	Nivel Intermedio - Polars Playground	<code>index, follow</code>
<code>avanzado.html</code>	Nivel Avanzado - Polars Playground	<code>index, follow</code>
<code>desafio.html</code>	Desafío - Polars Playground	<code>index, follow</code>
<code>lecciones_*.html</code>	Lecciones [Nivel] - Polars Playground	<code>noindex, nofollow</code>

Estructura de Datos JSON-LD

```
{
  "@context": "https://schema.org",
  "@type": "WebApplication",
  "name": "Polars Playground",
  "description": "Plataforma web interactiva para aprender Polars de forma práctica en el navegador.",
  "applicationCategory": "EducationalApplication",
  "operatingSystem": "Web Browser",
  "offers": { "@type": "Offer", "price": "0", "priceCurrency": "USD" },
  "inLanguage": "es"
}
```

Notas Técnicas e Implicaciones de WASM

- **Pyodide + Polars:** Carga Polars desde el lockfile de Pyodide v0.27.7. La función `p1.scan_csv()` no está soportada en WebAssembly debido a restricciones de I/O de Rust (genera panic); se usa `df.lazy()` como alternativa equivalente.
- **Fechas y Regex:** `p1.date_range()` requiere objetos `datetime.date` nativos. Las operaciones de `str.replace()` utilizan expresiones regulares por defecto, requiriendo escape para caracteres especiales.
- **Service Worker (v5):** Estrategia *network-first* que cachea recursos clave (Pyodide, CDN Monaco, CSS/JS) para ofrecer respuesta rápida y soporte en conexiones inestables.